ECE 562 Assignment 3: PPO II

Emmanuel Oluwasegun Olateju (eolateju@ualberta.ca)

October 19, 2026

Section 1: Theory

- $\pi_{old}(a|s) > 0 \implies \pi_\theta(a|s) > 0$
  $\pi_{old}(s) > 0 \impliedby d^{\pi_\theta}(s) > 0$
- p_0: Initial state distribution
  $p(s'|s,a)$: transition kernel
- **State-visitation distribution of a policy:**

$$d^{\pi_\theta}(s) \;=\; (1-\gamma)\sum_{t=0}^{\infty}\gamma^t\, p(S_t = s \mid s_0 \sim \rho_0, \pi)$$

- **Policy Gradient Theorem**

$$\nabla_\theta J(\theta) \;=\; \frac{1}{1-\gamma}\mathop{\mathbb{E}}_{\substack{s\sim d^{\pi_\theta}\\ a\sim\pi_\theta}}\Big[\nabla_\theta \log \pi_\theta(a|s)\, \hat{A}^{\pi_\theta}(s,a)\Big]$$

- **Per-step importance ratio:**

$$r_\theta(s|a) \;=\; \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$$

Q1

- Trajectories for a fixed length:

$$\tau = (s_0, a_0,\; s_1, a_1, \ldots, s_{T-1},\; a_{T-1},\; s_T)$$

- Reward is dependent only on (S_t, a_t, S_{t+1}):

$$R(\tau) \;=\; \sum_{t=1}^{T}\gamma^t r_t$$

- **Finite Horizon Objective**

$$J_T(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_\theta}\big[R(\tau)\big] \;=\; \sum_{\tau} p_\theta(\tau)\, R(\tau)$$

- **Trajectory Probability**

$$p_\theta(\tau) \;=\; p_0(S_0)\prod_{t=0}^{T-1}\pi_\theta(a_t|S_t)\,p(S_{t+1}|S_t,a_t)$$

(a)

$$J_T(\theta) \;=\; \sum_\tau\Big(\Big(p_0(S_0)\cdot\prod_{t=0}^{T-1}\pi_\theta(a_t|S_t)\,p(S_{t+1}|S_t,a_t)\Big)\cdot\sum_{t=0}^{T-1}\gamma^t r_t\Big)$$

$$J_T(\theta) \;=\; \sum_\tau p_\theta(\tau)\cdot R(\tau) \;=\; \sum_\tau \frac{p_{\theta_{\text{old}}}(\tau)\cdot p_\theta(\tau)}{p_{\theta_{\text{old}}}(\tau)}\cdot R(\tau)$$

Using the support assumption:

$$p_{\theta_{\text{old}}}(\tau) > 0 \;\longleftarrow\; p_{\theta_*}(\tau) > 0$$

Then:

$$\therefore \quad J_T(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\frac{p_\theta(\tau)\cdot R(\tau)}{p_{\theta_{\text{old}}}(\tau)}\right]$$

(b)

$$\frac{p_\theta(\tau)}{p_{\theta_{\text{old}}}(\tau)} \;=\; \frac{p_0(S_0)\prod_{t=0}^{T-1}\pi_\theta(a_t|S_t)\,p(S_{t+1}|S_t,a_t)}{p_0(S_0)\prod_{t=0}^{T-1}\pi_{\theta_{\text{old}}}(a_t|S_t)\,p(S_{t+1}|S_t,a_t)} \;=\; \prod_{t=0}^{T-1}\frac{\pi_\theta(a_t|S_t)}{\pi_{\theta_{\text{old}}}(a_t|S_t)}$$

The ratio above assumes that the initial state is constant regardless of policy, and the environment transition dynamics is independent of policy. In a model-free RL context, this assumption is necessary as it guarantees that appropriate modifications of current policy π_θ alone will optimize the objective function $J_T(\theta)$. Should the initial state and the transition dynamics be dependent on the policy, then some extra approach of reconciling the transition dynamics under the current policy and the behavioural policy needs to be employed, so optimizing or maximizing the current policy maximizes the objective function (the probability of taking the optimal trajectory). These assumptions make it possible to optimize our policy without knowing or modelling the transition dynamics and initial state probabilities.

(c) Over long horizons of multiplying $r_\theta(s_t,a_t)$ given as:

$$r_\theta(s_t,a_t) = \frac{\pi_\theta(s_t,a_t)}{\pi_{\theta_{\text{old}}}(s_t,a_t)}$$

$$\prod_{t=0}^{T-1}\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \;=\; \prod_{t=0}^{T-1} r_\theta(s_t,a_t)$$

There is no guarantee that the trajectories (action-sequences) generated by the current policy (π_θ) will match that of the behavioural policy ($\pi_{\theta_{\text{old}}}$) at all points in time into the future.

There is no guarantee that at some point into the future, the ratio $r_\theta(s_t, a_t)$ will not deviate significantly from 1, and the product across time may explode or vanish to zero creating high variance in gradient updates. PPO rather uses the single-step importqance ratio rather than the long trajectory product to minimize the variance of the gradient updates and make the ration $r_\theta(s_t, a_t)$ more stable.

Q2

- $s \sim d^{\pi_{\text{old}}}(s)$
  $a \sim \pi_{\theta_{\text{old}}}(a|s)$
- Full state-action distribution ratio:

$$\frac{d^{\pi_\theta}(s)\ \pi_\theta(a|s)}{d^{\pi_{\theta_{\text{old}}}}(s)\ \pi_{\theta_{\text{old}}}(a|s)} \ = \ w_\theta(s)\, r_\theta(s|a)\ ; \quad w_\theta(s) \ = \ \frac{d^{\pi_\theta}(s)}{d^{\pi_{\theta_{\text{old}}}}(s)}$$

(a) The single-step ratio $\pi_\theta(s|a)$ rather optimizes the conditional action probability $\pi_\theta(a|s)$ given the state s, which only accounts for how the policy changes the probability of visiting state s changing the policy changes how often a state (s_{t+1}) is visited given another state (s_t). This changes the probability of visiting certain states. Using the data generated from older (stale) policy $(\pi_{\theta_{\text{old}}})$ means that $d^{\pi_\theta}(s)$ will be very different from $d^{\pi_{\theta_{\text{old}}}}(s)$, and the current policy might ber updated on states that are rarely visited, or not updated on states that are visited often. As such correction of the ratio $\omega_{theta}(s)$ becomes important.

(b)
- Total Variation Distance:

$$D_{TV}(P\|Q) \ = \ \frac{1}{2}\sum_a |p(a) - q(a)|$$

- Known Bound:

$$\frac{\partial}{\partial\theta_i} D_{KL}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) \ \approx \ \sum_s \left[\frac{\partial}{\partial\theta_i}\mathbb{E}\right] \mathbb{E}[\pi_{\theta_{\text{old}}}(s) \cdot \pi_{\theta_{\text{old}}}(s)]$$

The upper bound on stat-visitation mismatch $||d^{\pi_\theta} - d^{\pi_{\theta_{\text{old}}}}||$, is directly proportional to D_{TV} (the total variation distance between action distributions given the same distance). If $\pi_{theta} \approx \pi_{\theta_{\text{old}}}$, then $D_{TV} \approx 0$ whcih forces $d^{\pi_\theta}(s) = d^{\pi_{\theta_{\text{old}}}}(s)$, and $\omega_\theta(s) = 1$, therefore limiting the overall state-visitation mismatch.

The PPO clipped surrograte objectibve also helps keep $r_\theta(s, a)$ within $[1 - \epsilon, 1 + \epsilon]$ ensuring that the online (current) policy does not change too much. This prevents large steps in action probabilities and implicitly controls the state-visitation mimatch.

The upper bound on the state-visitiation mismatch is however not a mathematical guarantee on the total variation distance over all unboserved action and states. It only guarantes a bound on all actions for a given state.

Q3: The Fully Corrected Off-Policy Gradient

(a) Policy gradient theorem with steps:

$$\nabla_\theta J(\theta) \ = \ \frac{1}{1-\gamma}\mathbb{E}_{s,a}[\nabla_\theta \log \pi_\theta(a|s)\, A^{\pi_\theta}(s, a)]$$

Starting from:

$$\frac{1-\gamma}{1-\gamma}\sum_s d^{\pi_\theta}(s) \;\longrightarrow\; \text{(continuing)}$$

$$\begin{aligned}
\nabla_\theta J(\theta) &= \frac{1}{1-\gamma}\sum_s d^{\pi_\theta}(s)\sum_a \pi_\theta(a|s)\,\nabla_\theta \log \pi_\theta(a|s)\,A^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\sum_s d^{\pi_\theta}(s)\sum_a \nabla_\theta \pi_\theta(a|s)\,A^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\sum_s d^{\pi_\theta}(s)\sum_a \pi_\theta(a|s)\,\nabla_\theta \log \pi_\theta(a|s)\,A^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d^{\pi_\theta},\,a\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s)\,A^{\pi_\theta}(s,a)]
\end{aligned}$$

Then using importance sampling:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \frac{1}{1-\gamma}\sum_s d^{\pi_{\text{old}}}(s)\frac{d^{\pi_\theta}(s)}{d^{\pi_{\text{old}}}(s)}\sum_a \pi_{\theta_{\text{old}}}(a|s)\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}\nabla_\theta \log \pi_\theta(a|s)\,A^{\pi_\theta}(s,a) \\
&= \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d^{\pi_{\text{old}}},\,a\sim\pi_{\theta_{\text{old}}}}[w_\theta(s)\cdot \hat{r}_t(\theta)\cdot \nabla_\theta \log \pi_\theta(a|s)\,A^{\pi_\theta}(s,a)]
\end{aligned}$$

(b) **Approximation PPO makes relative to (a) above:**

- PPO assumes that the probability of states visitation under the old policy (behavior policy) and the new (current/target) policy do not change significantly:

  $$d^{\pi_\theta}(s) \;\approx\; d^{\pi_{\theta_{\text{old}}}}(s) \quad\Rightarrow\quad w_\theta(s) \;\approx\; 1$$

  PPO assumes that updates to the target/current policy does not significantly change the trajectory under the current policy from that observed with the old policy.

- PPO firther assuems that the advantage of an action in a given state as evaluated by tge target policy is approximately the same as that evaluated by the behavior policy:

  $$\therefore A^{\pi_\theta}(s,a) \approx A^{\pi_{\theta_{\text{old}}}}(s,a)$$

  PPO therefore uses GAE to calculate $\hat{A}(s,a) = A^{\pi_\theta}(s,a) = A^{\pi_{\theta_{\text{old}}}}(s,a)$ using the roll out from the old $\pi_{\theta_{\text{old}}}$ samples.

When $\theta = \theta_{\text{old}}$:

$$\begin{gathered}
\pi_\theta(a|s) = \pi_{\theta_{\text{old}}}(a|s) \\
r_\theta(s,a) = 1 \\
\omega_\theta(s) = \frac{d^{\pi_\theta}(s)}{d^{\pi_{\theta_{\text{old}}}}(s)} = 1 \\
A^{\pi_\theta}(s,a) = A^{\pi_{\theta_{\text{old}}}}
\end{gathered}$$

Then:

$$J(\theta) = \sum_{s,a} d^{\pi_\theta}(s)\pi_\theta(s,a)R(s,a)$$
$$J(\theta_{\text{old}}) = \sum_{s,a} d^{\pi_{\theta_{\text{old}}}}(s)\pi_{\theta_{\text{old}}}(s,a)R(s,a)$$
$$\therefore J(\theta) = J(\theta_{\text{old}})$$

$$\therefore \quad \nabla_\theta J(\theta) \;\approx\; \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_{\text{old}}},\, a\sim\pi_{\theta_{\text{old}}}}[\hat{r}_t(\theta)\cdot\nabla_\theta \log \pi_\theta(a|s)\, A^{\pi_\theta}(s,a)]$$

Why is GAE with a learned critic still only an estimate of $A^{\pi_{\theta_{\text{old}}}}(s,a)$:
A learned critic still use sthe smaples of $\pi_{\theta_{\text{old}}}$ in its GAE estimation

$$\therefore V_\psi = V^{\pi_{\theta_{\text{old}}}}$$

and the advanrage function is assumed to be identical or close beacause $\pi_\theta(s,a) \approx \pi_{\theta_{\text{olds}}}(s,a)$:

$$A^{\pi_\theta}(s,a) = A^{\pi_{\theta_{\text{old}}}}$$

Q4

- $d^\pi(s') \;=\; (1-\gamma)p_0(s') + \gamma\sum_s d^\pi(s)\sum_a \pi(a|s)p(s'|s,a)$
- $p_\pi(s,s') = \sum_a \pi(a|s)p(s'|s,a)$
- $d^{pi}(s') = (1-\gamma)(I-\gamma p_\pi^T)^{-1}p_0$

(a)
- In a model free environment, the transition dynamics ($p(s'|s,a)$) are unknown making it impossible to compute $p_\pi(s,s')$
- Also the state-space is continuous makingit impossible to compute $(I-\gamma p_\pi^T)^{-1}$

(b) changing π_θ at any one state while collecting data can change the transition dynamics at that point in time and firther change the trajectroy downstream. this effectively modifies the state visitation frequencies across distinct future states.

Without $\omega_\theta(s)$, the state-distribution, PPO gradient weight are proportional to how $\pi_{\theta_{\text{old}}}$ visited states, creating an off-policy bias. However, the single step ration update and the upper bound on state-visitation mismatch ensures that gradient weights from $\pi_{\theta_{\text{old}}}$ are still relevant to π_θ at each step.

Section 2: Experiments and Report (Setup)

All experiments use the provided PPO implementation on `Hopper-v5` with the default hyperparameters (rollout length 2048, mini-batch 64, 10 epochs, learning rate 3×10^{-4}, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, entropy coefficient 0, value coefficient 0.5, gradient clipping 0.5). Every configuration was trained for 3×10^5 environment steps ($\lfloor 300000/2048 \rfloor = 146$ PPO iterations, 299,008 steps) with seeds $\{1,2,3\}$. All 15 runs completed; none triggered the numerical-instability stop. Every curve shows the mean across seeds with a shaded ±1 standard deviation band against environment steps. The evaluation return is that of the deterministic policy mean over 5 episodes, computed every 5 iterations. The exact commands are in `code/README.md`:

```
JOBS=12 bash run_all.sh     # 15 runs -> runs/*.csv
python plot.py              # -> figs/*.png
```

The diagnostic `kl` is the estimator $\widehat{\mathrm{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) = \hat{\mathbb{E}}_{(s,a)\sim\pi_{\theta_{\text{old}}}}[(r_\theta - 1) - \log r_\theta]$, computed on all 2048 samples of the rollout used for the update. Here $\pi_{\theta_{\text{old}}}$ is the policy that *collected* that rollout.

config	final eval	last-5 eval	first eval ≥ 1000	kl_start	clipfrac_start	kl_end	entropy
$k = 0$, clip	2106 ± 473	1924 ± 688	92k–102k	0	0	0.013	3.2
$k = 2$, clip	2060 ± 294	1822 ± 327	113k–225k	0.126	0.57	0.014	3.4
$k = 4$, clip	1578 ± 588	1552 ± 405	215k–256k	0.111	0.56	0.013	3.6
$k = 2$, ratio	256 ± 36	220 ± 44	never	11.9	0.97	16.6	-0.9
$k = 2$, naive	-1998 ± 784	-1882 ± 752	never	2.9×10^{12}	1.00	2.3×10^{12}	-17.3

Table 1: Summary over 3 seeds. "Final eval" is the last evaluation and "last-5 eval" the mean of the last five evaluations (mean $\pm$ std across seeds). "First eval ≥ 1000" gives the range over seeds of the first evaluation reaching a return of 1000. The KL, clip-fraction and entropy columns are averages over the second half of training ($> 1.5 \times 10^5$ steps). Entropy is that of the 3-D Gaussian policy.

Q5: How Stale Can PPO's Data Be?

(a) Modification

Implementation.

- **Staleness parameter** (`train.py`): A new flag `-k/--staleness` (default 0) is added. The training loop keeps a history of rollouts in a deque (line `169`):

  ```
  history = collections.deque(maxlen=k+1)
  ```

  At every PPO iteration i:

  1. A fresh rollout is collected with the current policy π_{θ_i} and appended to `history`, so every run uses the same number of environment steps regardless of k.
  2. The update uses the first rollout in the deque (line `178`):

     ```
     data = history[0]
     ```

     Once the deque is full, this is the rollout collected at iteration $i - k$ by $\pi_{\theta_{i-k}}$. Before the deque is full (iterations $i \leq k$), it is the oldest stored rollout. The deque discards older rollouts automatically, so exactly the $k + 1$ most recent rollouts are kept. With $k = 0$, the code reduces exactly to the original PPO.

- **Value re-evaluation** (`buffer.py`): A new method `refresh_values(ac)` of the `RolloutBuffer` class (lines `47-55`) recomputes the state values $V_\phi(s_t)$ for all 2048 stored states and the bootstrap value $V_\phi(s_T)$ with the current critic. It runs under `torch.no_grad()` (line `47`), so no gradients are tracked through these values. The training script calls it right before `compute_gae` (lines `57-79`).
  Observations, actions, rewards, done flags and the behavior log-probabilities $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ are not touched, so the denominator of r_θ is the density of the policy that actually generated each action.

- **Diagnostics** (`train.py`): A function `ratio_stats(...)` (lines `109-126`) is called on the selected stale rollout `data` before (line `185`) and after (line `199`) the 10 epochs, so all logged KL, ratio-percentile and clip-fraction values refer to the batch actually used.

Additionally, the update is wrapped in a `try` block (lines `183-208`). If the loss or KL becomes non-finite, or the Gaussian receives NaN parameters, training stops, the rows so far are kept, and a `.unstable` file records the step. This never triggered and is therefore not discussed further.

Why re-evaluate the values? GAE is built from the TD errors, and the critic targets are built from the advantages:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_{t+1}) - V(s_t), \qquad R_t = \hat{A}_t + V(s_t).$$

The values stored in the rollout are $V_{\phi_{i-k}}(s_t)$, computed by the critic at the time the data were collected. The current critic has since been trained for k more iterations ($10k$ epochs). Using the stored values as they are would:

- Compute advantages against an outdated baseline. That baseline approximated $V^{\pi_{\theta_{i-k-1}}}$, i.e. it was already one iteration behind when the data were collected, so the advantage bias grows with k.
- Regress the current critic towards targets formed from an old critic, undoing k iterations of value learning.

The critic is not a property of the data. It is our current best estimate of the value function, and it costs nothing to re-query it on the stored states. So the most accurate available V should be used.

Why must $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ not be recomputed? The ratio $r_\theta = \pi_\theta(a \mid s)/\pi_{\theta_{\text{old}}}(a \mid s)$ is an importance weight: it reweights actions sampled from the old policy so that, in expectation, they behave like actions sampled from the current policy. For this to work, the denominator must be the density under which each action was actually sampled, namely $\pi_{\theta_{i-k}}$. Recomputing it with the current network would replace the denominator by π_{θ_i}, the same as the numerator, giving $r_\theta \equiv 1$ at the start of every update. This falsely declares the data on-policy. It would:

- remove the only correction PPO has for the action distribution (Q2a);
- make `kl_start` and `clipfrac_start` identically zero;
- centre the clipping region on a policy that never generated the data.

Unlike V, the behavior log-probability is a fixed fact about how the data were generated, not an estimate to be improved.

Is the re-evaluated GAE on-policy for π_{θ_i}? No. Re-evaluation only changes the baseline. The states $s_t \sim d^{\pi_{\theta_{i-k}}}$, the actions $a_t \sim \pi_{\theta_{i-k}}$ and the rewards along the trajectory all come from the old policy. The λ-return inside GAE sums rewards collected by $\pi_{\theta_{i-k}}$ over many future steps, without any per-step correction. So $\hat{A}$ is (at best) an estimate of $A^{\pi_{\theta_{i-k}}}$, not $A^{\pi_{\theta_i}}$. Moreover, the current critic approximates neither exactly, since it is trained on a mixture of stale rollouts. The state-distribution mismatch $w_\theta(s)$ (Q2) is also untouched.

(b) Results

Observations.

- **Return** (Fig. 1, Table 1). Learning is clearly delayed by staleness. With $k = 0$ all seeds pass a return of 1000 at about 1×10^5 steps. With $k = 2$ this happens between 1.1×10^5 and 2.25×10^5, and with $k = 4$ only after 2.1–2.6 $\times 10^5$ steps. At $k = 2$ the final return (2060 ± 294) catches up with $k = 0$ (2106 ± 473), but the area under the curve is much smaller. At $k = 4$ the final return is lower (1578 ± 588) and the curve is still rising.
- **`kl_start` and `clipfrac_start`** (Fig. 2). For $k = 0$ both are exactly zero, a sanity check that the update starts on-policy ($r_\theta \equiv 1$). For $k > 0$ both grow during the first $\sim 1.5 \times 10^5$ steps

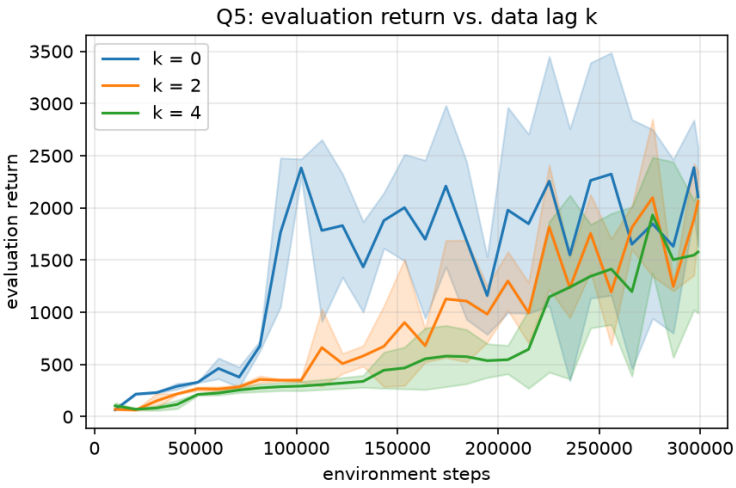
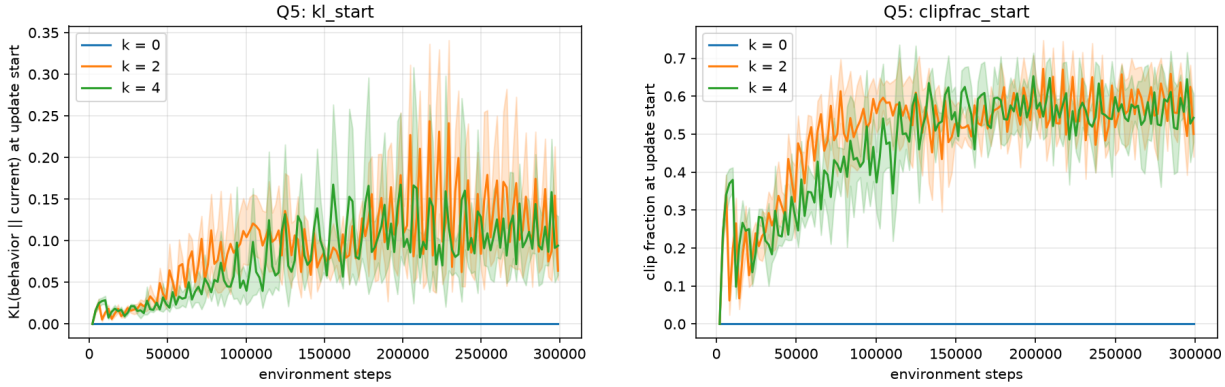


Figure 1: Q5: evaluation return for $k \in \{0, 2, 4\}$ (clip loss).



(a) `kl_start` (b) `clipfrac_start`

Figure 2: Q5: mismatch between the behavior policy and the current policy at the start of each update.

and then plateau at `kl_start` ≈ 0.1–0.13 and `clipfrac_start` ≈ 0.55–0.6. In the second half, the ratios at the start of the update are far from 1: the 5th/95th percentiles of r_θ are 0.39/1.83 ($k = 2$) and 0.41/1.81 ($k = 4$). So well over half the batch is outside $[1 - \epsilon, 1 + \epsilon]$ before a single gradient step.

- **Unexpected result.** We expected $k = 4$ to show a larger start-of-update mismatch than $k = 2$, but the two plateaus overlap (0.111 vs. 0.126, Table 1). The explanation is in the end-of-update diagnostics: `kl_end` ≈ 0.013 for every k, *smaller* than `kl_start`. So, measured against the stale behavior policy, each update moves the policy back towards $\pi_{\theta_{i-k}}$ (see (c)). Because every update re-anchors the policy to old data, the total drift between $\pi_{\theta_{i-k}}$ and π_{θ_i} does not grow linearly with k; it saturates. In addition, $k = 4$ learns more slowly, so its policy moves less per iteration, and keeps a larger entropy (3.6 vs. 3.4). A wider Gaussian produces a smaller KL for the same mean shift.

(c) Analysis

How stale can the data be? In this setup, performance already degrades at $k = 2$: it reaches a return of 1000 roughly 1.2–2.4× later than $k = 0$, with larger seed variance early on. The degradation becomes substantial at $k = 4$, which ends lower and is still far from converged at 3×10^5 steps. Only $k = 0$ (truly on-policy) is free of the effect. PPO tolerates a small lag in the sense that it does not diverge, which is clipping at work. But any lag that makes r_θ leave the trust region at the start of the update means more environment steps are needed to reach the same return, i.e. lower sample efficiency.

Explanation using Q2. Even with exact advantages, the correctly reweighted gradient needs the full factor $w_\theta(s)\, r_\theta(s,a)$, where $w_\theta(s) = d^{\pi_\theta}(s)/d^{\pi_{\theta_{\text{old}}}}(s)$. PPO uses only r_θ and implicitly sets $w_\theta(s) \approx 1$. The bound $\|d^{\pi_\theta} - d^{\pi_{\theta_{\text{old}}}}\|_1 \leq \frac{2\gamma}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_{\theta_{\text{old}}}}}[D_{TV}]$ says this approximation is only justified while the two policies are close. With $\gamma = 0.99$ the factor in front is $2\gamma/(1-\gamma) = 198$, and the left-hand side can never exceed 2. So the bound only tells us something when the average D_{TV} is below about 0.01. With `kl_start` ≈ 0.12 and more than half of the ratios outside $[0.8, 1.2]$, the policies are far from that close, and the bound gives no guarantee that the state distributions are similar. The data are therefore drawn from $d^{\pi_{\theta_{i-k}}}$, which may differ substantially from $d^{\pi_{\theta_i}}$, and the gradient puts weight on states the current policy no longer visits and too little on states it does visit. In addition, $\hat{A} \approx A^{\pi_{\theta_{i-k}}} \neq A^{\pi_{\theta_i}}$ (Q3b), a second error that also grows with the policy gap.

Explanation using Q4. w_θ cannot be computed because $d^\pi = (1-\gamma)(I - \gamma P_\pi^\top)^{-1} p_0$ depends on the unknown transition kernel and, through the matrix inverse, on the policy at *all* states. Changing the policy at one state changes P_π in that row, and the inverse spreads the change to every state reachable from it. In Hopper, slightly different joint torques early in an episode lead to different gaits, heights and fall times, so after k iterations of learning, the states visited by $\pi_{\theta_{i-k}}$ (e.g. falling after a short hop) are systematically different from those of π_{θ_i} (longer hopping). The stale rollout's states are simply the wrong states to improve the current policy on, and nothing in the loss corrects for this.

The clip fraction and the fraction of useful samples. For a sample with ratio r and advantage $\hat{A}$, the clipped objective (maximised by PPO) is

$$\min\big(r\hat{A},\ \text{clip}(r, 1-\epsilon, 1+\epsilon)\hat{A}\big).$$

If r is within $[1-\epsilon, 1+\epsilon]$, i.e. $|r-1| \leq \epsilon$, both branches are equal and the gradient is $\hat{A}\nabla_\theta r$, which is non-zero whenever $\hat{A} \neq 0$. So every in-range sample contributes. If r is out of range ($|r-1| > \epsilon$), there are two cases:

- $r > 1+\epsilon,\ \hat{A} > 0$ or $r < 1-\epsilon,\ \hat{A} < 0$: the min selects the *clipped* branch, which is constant in θ, so the gradient is zero. The policy has already moved in the direction the advantage asks for.
- $r > 1+\epsilon,\ \hat{A} < 0$ or $r < 1-\epsilon,\ \hat{A} > 0$: the min selects the *unclipped* branch $r\hat{A}$, so the gradient $\hat{A}\nabla_\theta r$ is non-zero. It pushes r back towards 1, i.e. the current policy π_{θ_i} towards the stale policy $\pi_{\theta_{i-k}}$.

The indicator $\mathbf{1}[|r_i - 1| > \epsilon]$ marks sample $i \in \{1, \ldots, N\}$ as out of range, so `clipfrac` $= \frac{1}{N}\sum_i \mathbf{1}[|r_i - 1| > \epsilon]$ is the fraction of out-of-range samples, and $1 -$ `clipfrac` is the fraction of in-range samples, all of which contribute a gradient. This is only a lower bound because out-of-range samples in the second case above also contribute a non-zero gradient but are not counted.

With stale data this case is common, since $\hat{A}$ was estimated without regard to the direction in which the policy has moved over the last k iterations. Also, `clipfrac_start` is only a snapshot: during the 10 epochs some samples enter and leave the clipping region.

Our results show why this matters. For $k \in \{2, 4\}$, $1 - \texttt{clipfrac_start} \approx 0.43$, so fewer than half the samples are guaranteed to give a normal policy-gradient signal. Many of the remaining non-zero gradients come from the second case above and pull the policy back towards the stale behavior policy. This matches the drop from `kl_start` ≈ 0.12 to `kl_end` ≈ 0.013 and from `clipfrac` ≈ 0.57 to 0.18. Much of each update is spent pulling the policy back into a trust region centred on an outdated policy, rather than improving the current one. This explains the slower learning in Fig. 1.

Why PPO discards its data after every iteration. Reusing a rollout after the policy has moved has three effects:

1. The number of samples with an informative clipped gradient shrinks ($1 - \texttt{clipfrac}$ falls from 1 to ≈ 0.43).
2. The trust region is centred on the wrong policy, so updates partly undo progress.
3. The uncorrected state-distribution mismatch $w_\theta(s)$ and advantage mismatch $A^{\pi_\theta} \neq A^{\pi_{\theta_{\text{old}}}}$ grow with the policy gap and bias the gradient.

Clipping keeps the procedure stable but cannot remove any of these effects. A fresh rollout costs only 2048 environment steps, and it restores $r_\theta \equiv 1$, $w_\theta \equiv 1$ and $A^{\pi_\theta} = A^{\pi_{\theta_{\text{old}}}}$ at the start of the update. Reusing older data instead buys nothing (in our runs every k collects the same number of steps) and adds the three effects above. So PPO stays (almost) on-policy: it reuses each rollout only for the 10 epochs of one iteration, where the policy is kept close by design.

Q6: Does the Importance Ratio Help with Stale Data?

(a) Modification and equality of gradients

Implementation (`ppo.py`). `PPOAgent` takes `loss_type`$\in$ {`naive`, `ratio`, `clip`}, selected with `--loss` in `train.py`. With `ratio = exp(newlogprob - b_logprobs)` and the normalized advantages:

$$\begin{aligned}
\text{naive:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}[\log \pi_\theta(a \mid s)\, \hat{A}], \\
\text{ratio:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}[r_\theta(s,a)\, \hat{A}], \\
\text{clip:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}\big[\min\big(r_\theta \hat{A},\ \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)\hat{A}\big)\big].
\end{aligned}$$

The value loss, gradient clipping, advantage normalization (once per rollout) and the stored behavior log-probabilities are identical for all three.

Equal gradients at $r_\theta = 1$. $\hat{A}$ and $\log \pi_{\theta_{\text{old}}}(a \mid s)$ do not depend on θ. Since $r_\theta = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$,

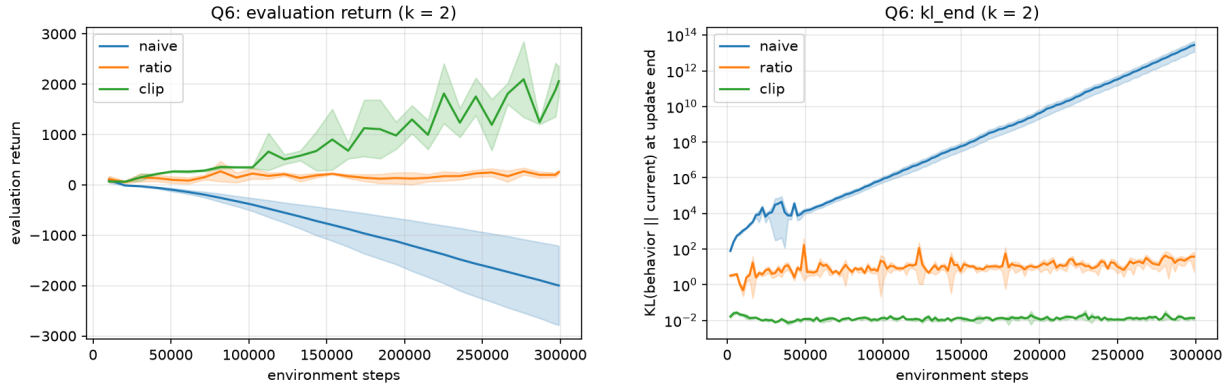
$$\nabla_\theta r_\theta(s,a) = r_\theta(s,a)\, \nabla_\theta \log \pi_\theta(a \mid s).$$

- Naive: $\nabla_\theta L_{\text{naive}} = -\hat{\mathbb{E}}[\hat{A}\, \nabla_\theta \log \pi_\theta]$.
- Ratio: $\nabla_\theta L_{\text{ratio}} = -\hat{\mathbb{E}}[\hat{A}\, r_\theta\, \nabla_\theta \log \pi_\theta]$, which equals the naive gradient when $r_\theta = 1$ for every sample.

- Clip: at $r_\theta = 1$ we have $1-\epsilon < r_\theta < 1+\epsilon$ (for $\epsilon > 0$), so $\text{clip}(r_\theta, 1-\epsilon, 1+\epsilon) = r_\theta$ in a neighbourhood of this point. Both arguments of the min coincide and equal $r_\theta \hat{A}$, so the per-sample objective is locally $r_\theta \hat{A}$ and its gradient is $\hat{A} \nabla_\theta r_\theta = \hat{A}\, r_\theta \nabla_\theta \log \pi_\theta = \hat{A} \nabla_\theta \log \pi_\theta$.

So all three losses have the gradient $-\hat{\mathbb{E}}[\hat{A}\, \nabla_\theta \log \pi_\theta(a \mid s)]$, the standard on-policy policy gradient. This is exactly the situation at the first mini-batch of an update with $k = 0$. The losses differ only once $r_\theta \neq 1$, i.e. for stale data or after the first gradient step.

(b) Results



(a) evaluation return

(b) `kl_end` (log scale)

Figure 3: Q6: the three surrogate losses with $k = 2$. The clip curve is the $k = 2$ run from Q5.

- **clip** learns normally (final 2060 ± 294) with `kl_end` ≈ 0.01–0.02 throughout.
- **ratio** makes almost no progress. The return stays at 100–400 (final 256 ± 36), about the level of a policy that stands and falls. Its `kl_end` is $\sim$ 1–100 (mean 16.6 in the second half), three orders of magnitude above clip, and 97% of samples end the update outside the trust region. The median ratio at the end of an update is 0.01 and the entropy becomes negative ($4.2 \to -2.2$ for seed 1), i.e. the policy collapses.
- **naive** is the worst. The return decreases monotonically from the first evaluation and reaches -1998 ± 784. Its `kl_end` is already ≈ 90 after the first update and then grows *exponentially* (a straight line on the log plot) to $\sim 10^{13}$. The entropy falls to ≈ -17 (-23 for seed 1, i.e. $\log \sigma \approx -9$ per action dimension).
- **Numerical stability.** No run produced a non-finite loss or KL, so all 146 iterations are available for every configuration and no truncation was needed. The naive run is, however, numerically degenerate: its ratios underflow to 0 for essentially all samples (95th percentile $\approx 10^{-17}$) and its value loss is $\sim 10^5$.

(c) Analysis

Without the ratio (naive). The naive loss ignores where the data came from. It treats $(s, a) \sim \pi_{\theta_{\text{old}}}$ as if it were drawn from π_θ, so it lacks *both* corrections of Q3(a): $w_\theta(s)$ and $r_\theta(s, a)$. Even in expectation it is not the gradient of $J(\theta)$ once $\theta \neq \theta_{\text{old}}$. More damaging, it has no term that

anchors θ near θ_{old}, and the objective it maximizes, $\hat{\mathbb{E}}[\hat{A} \log \pi_\theta]$, is *unbounded above*. For a Gaussian policy,

$$\hat{A}_i \log \pi_\theta(a_i \mid s_i) = -\hat{A}_i \frac{\|a_i - \mu_\theta(s_i)\|^2_{\Sigma^{-1}}}{2} - \hat{A}_i \sum_j \log \sigma_j + \text{const.}$$

For a sample with $\hat{A}_i < 0$, this term grows without bound as $\mu_\theta(s_i)$ moves away from a_i. Shrinking σ multiplies that growth by $1/\sigma^2$. Positive-advantage samples bound their own term by 0 but cannot balance this. The $\log \sigma$ terms cancel on average because the advantages are normalized to zero mean. Ten epochs of Adam on this objective therefore drive $\sigma \to 0$ (entropy $\to -17$) and push the mean away from the actions that did badly, to arbitrarily large values. So the KL grows geometrically, at a rate set by the learning rate, as observed. Huge action magnitudes explain the *negative* returns: Hopper's control cost $10^{-3}\|a\|^2$ is computed on the raw action, so it dominates the healthy and forward rewards. The failure modes without the ratio are a biased gradient that is not the policy gradient of any nearby policy, combined with an unbounded objective that leads to divergence and entropy collapse. Note that the gradient is never weighted by how likely the sample is under the current policy, so a sample the current policy would never generate still produces a full-size gradient.

With the ratio but without clipping (ratio). Weighting by r_θ restores the action-distribution correction. In expectation, $\hat{\mathbb{E}}[r_\theta \nabla \log \pi_\theta \hat{A}]$ is the Q3(b) estimator, which corrects $a \sim \pi_{\theta_{\text{old}}}$ to $a \sim \pi_\theta$ but still assumes $w_\theta(s) \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$. This is why ratio does much better than naive (no divergence, positive return). But:

1. **Unbounded surrogate.** $r\hat{A}$ grows without bound as $r \to \infty$ for $\hat{A} > 0$, and the gain is bounded by 0 as $r \to 0$ for $\hat{A} < 0$. Optimizing it for 10 epochs concentrates probability mass on a few positive-advantage actions. The policy narrows (negative entropy), most stored actions become improbable (median $r_{\text{end}} = 0.01$), and a handful of samples with large r dominate the batch.
2. **High variance.** With r spread over several orders of magnitude, the importance-weighted gradient has a very large variance. Its effective sample size is a small fraction of 2048, so the gradient is noise-dominated, which explains the stalled return.
3. **Approximations break down.** Since `kl` ~ 10, the policies are far apart. Then $w_\theta(s) \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$ (Q2, Q3b) are badly violated: the surrogate $\hat{\mathbb{E}}[r\hat{A}]$ is only a first-order local approximation of $J(\theta) - J(\theta_{\text{old}})$, so maximizing it far from θ_{old} no longer improves the true return.

In short, the ratio fixes the bias in the action distribution, but without a constraint the optimizer exploits the surrogate in a region where it is meaningless and the importance weights have huge variance.

How clipping helps. The clipped objective is flat in r once r leaves $[1-\epsilon, 1+\epsilon]$ in the improving direction. This removes the incentive to push any sample's ratio further, which bounds the effective importance weights and the variance of the gradient. It keeps π_θ close to $\pi_{\theta_{\text{old}}}$, where $w_\theta \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$ are reasonable (the bound in Q2b). The pessimistic min also leaves a gradient on samples where the policy moved in the wrong direction, pulling them back. Empirically, `kl_end` $\approx$ 0.013 for every k, compared with ~ 17 (ratio) and $\sim 10^{12}$ (naive).

What clipping does not fix.

- It does not correct the state distribution: $w_\theta(s)$ is still set to 1 (Q2, Q4).
- It does not correct the advantages: $\hat{A}$ still estimates $A^{\pi_{\theta_{\text{old}}}}$ (with an approximate critic), not A^{π_θ} (Q3b).

- It does not remove the mismatch already present in stale data. The trust region is centred on $\pi_{\theta_{\text{old}}} = \pi_{\theta_{i-k}}$, not on the current policy, so at the start of an update 57% of samples are already clipped and part of the update is spent pulling the policy back towards an outdated policy (Q5c). This is why clip with $k > 0$ is stable but slower than $k = 0$.
- It does not enforce a hard trust region. Clipping only removes the gradient of individual samples; the policy can still move further through the shared network parameters, through other samples, or over many epochs. It gives no guarantee on D_{TV} or KL. That clip stays at $\texttt{kl} \approx 0.01$ is an empirical property of these hyperparameters (10 epochs, small learning rate, gradient clipping), not a guarantee.

Clipping therefore makes PPO *robust* to mildly off-policy data, but it does not make PPO an off-policy algorithm. The full correction $w_\theta(s)\, r_\theta(s, a)$ with A^{π_θ} is still missing, which is why PPO collects fresh data every iteration.